

Creating a voice-activated AI responder in Python

Cite as: Martin Paul Eve, "Creating a voice-activated AI responder in Python", *eve.gd: Martin Paul Eve*, September 24, 2023, <https://doi.org/10.59348/9rmf6-60g83>.



How do you solve a problem like a séance? With Python and GPT3, is my answer.

I've spent this weekend working on the technology for a forthcoming Artangel exhibition in Leeds that features an AI séance. The idea is that we want a voice-activated system that uses OpenAI's GPT engine to respond to the user as though they were in a seance, conversing with the spirit of Alicia Boole Stott (1860-1940), an English mathematician known for her significant work in four-dimensional geometry. Spooky tech!

I wanted to note a few of the technical challenges I faced with this project here.

The first is that we are running on a Raspberry PI. This meant that pyaudio struggled to keep up with the buffering as the unit is under-powered. There was also quite a bit of setup involved in getting a USB microphone to run input while outputting through the headphone jack. In the end I used this `/etc/asound.conf` configuration:

```

pcm.!default {
    type asym
    playback.pcm {
        type plug
        slave.pcm "output"
    }
    capture.pcm {
        type plug
        slave.pcm "dubler"
    }
}

ctl.!default {
    type hw
    card 0
}

pcm.output {
    type hw
    card 0
    device 0
}

pcm.dubler {
    type hw
    card 3
    device 0
}

```

The other major technical challenge was detecting when there is noise that we should process and when it's background gibberish that we should discard – or is speech, but not directed at our unit.

Our solution was to give start and end phrases that should bracket how you address the spirit: “Hello, Alicia. Can you tell me something about the fourth dimension? Spirit hear me.”

We also needed a silence/noise detection system, so the `check_voice_volume` function in [my main event loop](#) handles this. It basically looks for a consecutive 50 chunks of silence and, if it has previously found voice input over the noise threshold, appends this to the data buffer.

The silence threshold is very different on different devices with different microphones, so we have to tweak it according to the final hardware. In the meantime, I did a device selection procedure like this:

```
indices = [  
    (device_number, card_name)  
    for device_number, card_name in enumerate(alsaaudio.cards())  
    if "C930e" in card_name or "USB" in card_name  
]
```

This then allows us to construct an alsaaudio object thus:

```
recorder = alsaaudio.PCM(  
    type=alsaaudio.PCM_CAPTURE,  
    channels=CHANNELS,  
    rate=RATE,  
    format=INPUT_FORMAT,  
    periodsize=FRAME_SIZE,  
    device=f"hw:{indices[0][0]}",  
)
```

As noted above, the RPI wasn't powerful enough to keep up with pyaudio's buffer processing, but with massive thanks to [this StackOverflow answer](#) I was able to get the system running on alsaaudio.

One final gotcha: this system works by writing an output wave file. Different microphones have a different number of input channels, but the output channels in the wave need to match the input specification, or you will get a nasty error.